University of Montpellier

Master's Degree in Computer Science and Robotics

Internship Report

Academic Year 2025–2026

Design and Implementation of a Simulation and Visual Analysis Platform for Drone-Based Logistics Inspection

Prepared by: Rendra DJUNAEDI
Institution: University of Montpellier
Host company: Audensiel
Academic supervisor: Salih ABDELAZIZ
Company mentor: Chaouki TADJINE

Acknowledgements

I would first like to express my sincere gratitude to everyone who contributed, directly or indirectly, to the successful completion of this internship project. I would particularly like to thank my company mentor for the guidance, availability and advice provided throughout the internship. This support helped me progress both technically and methodologically, while strengthening my autonomy in carrying out the project.

I would also like to thank my academic supervisor for the follow-up, constructive feedback and support provided during this period. Finally, I thank the entire host team for their help, collaborative spirit and the enriching discussions that made this experience an important step in my professional development.

Abstract

This report presents the work completed during a master's internship devoted to the design, integration and functional assessment of a simulation and visual analysis platform for drone-based logistics inspection. The starting point was a preliminary three-dimensional environment initiated by the company mentor in Unreal Engine, in which AirSim and a simulated drone had already been integrated. The internship focused on extending this foundation into a coherent workflow connecting warehouse configuration, drone missions, multi-camera acquisition, web supervision and visual-data exploitation.

The main contributions include the structuring of racks and slots, the implementation of a generic mission reusable across several rack configurations, PID-based drone positioning, the acquisition of `cam0` and `cam1` views, UDP and Pixel Streaming integration, logistics-tag detection, optical character recognition and barcode decoding, and automatic annotation from simulation segmentation masks. Lighting and orientation variants were also generated to increase the diversity of the synthetic data used for computer-vision development.

Keywords: drone, Unreal Engine, AirSim, simulation, logistics inspection, OCR, UDP, Pixel Streaming, automatic annotation, synthetic data.

Contents

Acknowledgements **i**

Abstract **ii**

Contents **iii**

List of Figures **vi**

List of Tables **vii**

List of Acronyms **viii**

General Introduction **1**

1 Internship Overview **2**
1.1 Company Presentation 2
1.1.1 General Overview 2
1.1.2 Organization of the R&D Department 2
1.1.3 Innovation Lab 3
1.1.4 Company Projects 3
1.1.5 Available Equipment and Resources 4
1.1.6 Social and Environmental Commitments 4
1.2 Project Presentation 5
1.2.1 Context and Problem Statement 5
1.2.2 Project Objectives 6
1.2.3 Functional Scope 6
1.2.4 Technical and Operational Constraints 7
1.2.5 Internship Assignments 7

2 Methodology and Internship Progress **9**
2.1 Project Methodology and Organization 9
2.1.1 Kickoff Meeting and Requirement Framing 9

2.1.2 Task Planning 9
2.1.3 Working Method and Progress Tracking 9
2.1.4 Collaboration, Configuration and Version Control . . . 10
2.2 Internship Progress 11
2.2.1 Initial Technical Baseline and Analysis 11
2.2.2 Warehouse and Rack Configuration 13
2.2.3 Drone Missions and AirSim Integration 15
2.2.3.1 Mission Initialization and Rack Selection . . . 15
2.2.3.2 Slot Traversal Strategy 16
2.2.3.3 Target Pose Computation 16
2.2.3.4 PID-Based Positioning 16
2.2.3.5 Stabilization, Image Acquisition and Mission Outputs 17
2.2.4 Cameras, Streaming and Real-Time Rendering 18
2.2.4.1 UDP Streaming Performance Optimization . 19
2.2.4.2 MultiView Interface in Unreal Engine 21
2.2.5 Structuring the Back-End Application 22
2.2.6 Evolution of the Supervision Interface 23
2.2.7 Visual Tag Detection, OCR and Barcode Recognition . 26
2.2.8 Synthetic Dataset and Automatic Annotation Pipeline 27
2.2.8.1 Synthetic Logistics Tags and Confidentiality . 27
2.2.8.2 RGB Capture and Metadata Acquisition . . . 27
2.2.8.3 Segmentation-Based Automatic Annotations . 27
2.2.8.4 Lighting and Orientation Variants 28
2.2.8.5 Quality Controls and Current Limits 28
2.2.9 Functional Validation of the Integrated Workflow . . . 29
2.2.10 Summary of Personal Contributions 30

3 Feedback and Reflection **32**
3.1 Reflection 32
3.1.1 Project Contributions 32
3.1.2 Analysis of Technical Choices 32
3.1.3 Difficulties Encountered and Solutions 33
3.2 Skills Acquired 34
3.2.1 Technical Skills 34
3.2.2 Methodological Skills 34
3.3 Current Limitations 34
3.4 Future Project Evolution 35

General Conclusion and Outlook **36**

Bibliography **37**

List of Figures

2.1 Overall architecture connecting the React interface, the FastAPI services, AirSim, Unreal Engine and the visual-processing components. 11
2.2 Logical representation of a rack with three levels and four slots per level. 14
2.3 Global verification of the warehouse and spatial arrangement of the racks. 14
2.4 Local verification of racks, pallets and tags in the scene. 14
2.5 Sequential rack-inspection workflow from mission selection to capture, annotation and result storage. 18
2.6 Pallet and tag Render Targets used by the specialised views. . 19
2.7 Render Target used for the overhead warehouse view. 19
2.8 Principal Blueprints associated with the drone. 20
2.9 Simplified UDP-to-MJPEG streaming pipeline. 21
2.10 Structure, initialization and interaction logic of the MultiViewHUD widget. 22
2.11 Previous version of the supervision interface. 25
2.12 Final version of the supervision interface. 25
2.13 Visual analysis of a logistics tag in the supervision interface. The outer box identifies the complete tag, while internal regions show detected text and barcodes together with recognised values and confidence scores. 26
2.14 Simplified generation and exploitation chain for synthetic logistics tags and automatic annotations. 29

List of Tables

1.1 Main software and hardware resources used during the internship 5

2.1 Main development phases of the internship 10
2.2 Initial assets and work carried out during the internship 12
2.3 Role of the warehouse configuration 13
2.4 Principal parameters of the rack-inspection mission 15
2.5 Configuration of the AirSim inspection cameras 18
2.6 Main exchanges coordinated by the back end 23
2.7 Functional evolution of the interface 24
2.8 Synthetic capture categories generated for each occupied slot . 29
2.9 Functional validation of the integrated workflow 30
2.10 Summary of personal contributions 31

List of Acronyms

API	Application Programming Interface
FOV	Field of View
FPS	Frames Per Second
HTTP	Hypertext Transfer Protocol
HUD	Head-Up Display
JSON	JavaScript Object Notation
MJPEG	Motion JPEG
NED	North-East-Down
OCR	Optical Character Recognition
PID	Proportional–Integral–Derivative
RGB	Red, Green and Blue image format
TCP	Transmission Control Protocol
TTL	Time-To-Live
UDP	User Datagram Protocol
UI	User Interface

General Introduction

The increasing use of autonomous systems in logistics creates a need for controlled environments in which navigation, perception and inspection strategies can be developed and evaluated before being transferred to real operating conditions. Three-dimensional simulation provides a repeatable and risk-free framework for studying these systems while maintaining control over the environment, the robot state and the generated visual data.

At the beginning of the internship, the available starting point consisted of a preliminary 3D environment initiated by the company mentor in Unreal Engine. AirSim had already been integrated, and a simulated drone had been added and configured to fly within the scene. However, this initial prototype did not yet implement a complete warehouse inspection workflow. Rack-based missions, application orchestration, web supervision, multi-camera streaming, visual-result management and automatic dataset annotation still had to be designed, extended or consolidated.

The internship therefore focused on developing an integrated simulation and visual analysis demonstrator for logistics inspection. The work involved structuring the warehouse scene, implementing repeatable rack-inspection missions, coordinating the Unreal Engine/AirSim simulation with a FastAPI back end and a React interface, integrating several visualization channels, and preparing the data required for tag detection, OCR, barcode reading and model training. This report presents the adopted methodology, the technical contributions, the observed functional results, the current limitations and the main perspectives for future work.

Chapter 1

Internship Overview

1.1 Company Presentation

1.1.1 General Overview

Audensiel is a French technology consulting and digital engineering company founded in 2013. It specializes in digital transformation, software engineering, data engineering, artificial intelligence, cloud computing, cybersecurity, embedded systems and the Internet of Things. Over the years, the company has experienced steady growth and now supports major industrial and service-sector clients in the design, development and deployment of innovative technological solutions.

Audensiel operates in various business sectors, including aerospace, automotive, energy, healthcare, telecommunications, finance and transportation. By combining technical expertise with innovation, the company assists its clients throughout the entire lifecycle of their digital projects, from feasibility studies and software design to deployment and maintenance.

The internship presented in this report was carried out at Audensiel Nord, located in Boulogne-Billancourt, within the Research and Development department. This department focuses on the development of innovative technologies and proof-of-concept projects, particularly in the fields of artificial intelligence, computer vision, simulation and autonomous systems.

1.1.2 Organization of the R&D Department

Unlike traditional consulting activities that are mainly client-oriented, the Research and Development department focuses on internal innovation projects

and experimental technologies. The objective is to explore new technical solutions, evaluate their feasibility and develop demonstrators that may later become commercial products or industrial services.

Project development follows an iterative approach. Tasks are progressively assigned, followed by regular technical meetings where engineers present their progress, discuss implementation choices and receive feedback from senior engineers and project managers. This collaborative workflow encourages continuous improvement while maintaining flexibility throughout the development process.

During my internship, my supervisor defined the project's main objectives while allowing me significant autonomy in proposing technical solutions, implementing new features and contributing to architectural decisions. This working environment strengthened my technical skills, problem-solving abilities and capacity to work independently within a multidisciplinary team.

1.1.3 Innovation Lab

At the heart of the center is the Innovation Lab, whose mission is to explore emerging technologies and transform research ideas into operational prototypes and demonstrators. The laboratory develops proof-of-concept solutions that can later be integrated into industrial projects or proposed to clients as innovative services.

The project carried out during this internship is part of this innovation strategy. It combines drone simulation, computer vision, artificial intelligence and web technologies to develop an intelligent logistics inspection platform capable of reproducing realistic warehouse scenarios.

1.1.4 Company Projects

The Innovation Lab develops a wide range of research and development projects covering several technological domains.

Artificial intelligence and data science represent one of the laboratory's primary research areas, including Generative AI, machine learning, deep learning, computer vision and intelligent automation.

The laboratory also conducts projects involving embedded systems, robotics and the Internet of Things, focusing on connected devices, real-time data

processing and industrial automation.

Another important research area concerns autonomous systems, including drones, autonomous vehicles and simulation-based inspection platforms. These projects involve simulation technologies, perception systems, LiDAR, radar, computer vision and autonomous navigation algorithms.

Cybersecurity also plays an important role, with projects dedicated to securing embedded systems, cloud infrastructures, connected devices and industrial communication protocols.

These research activities allow Audensiel to experiment with emerging technologies, strengthen internal expertise and prepare innovative solutions that can later be deployed in industrial environments.

1.1.5 Available Equipment and Resources

The work was carried out using a software stack combining simulation, application development and computer-vision tools. Unreal Engine 4.27 was used for the 3D environment and real-time rendering, while Microsoft AirSim provided the drone-control and image-acquisition interfaces. Python was used for mission logic, data processing and back-end services, and React was used for the supervision interface.

A physical drone was available in the laboratory environment, but the developments and validations described in this report were carried out primarily with the simulated multirotor in AirSim.

1.1.6 Social and Environmental Commitments

Audensiel actively promotes Corporate Social Responsibility through initiatives addressing environmental sustainability, diversity and social inclusion.

The company implements various measures to reduce its environmental impact, including energy-efficient office facilities, responsible waste management, recycling programs and the adoption of sustainable mobility solutions.

Audensiel also encourages diversity and equal opportunities by supporting inclusive recruitment policies, professional development programs and initiatives promoting the integration of people with disabilities into the workplace.

Table 1.1: *Main software and hardware resources used during the internship*

Resource	Category	Role in the project
Unreal Engine 4.27	**Simulation**	Warehouse environment, real-time rendering, Blueprints, cameras and Render Targets.
Microsoft AirSim	**Simulation and control**	Multirotor control, object poses, camera acquisition and segmentation images.
Python and FastAPI	**Application services**	Mission execution, orchestration, streaming services, metadata and result management.
React	**Web application**	Session control, rack selection, mission monitoring, camera views and result consultation.
OpenCV	**Computer vision**	Image conversion, resizing, JPEG encoding and photometric transformations.
SolidWorks	**3D modelling**	Preparation or adaptation of 3D assets when required.
Development workstation	**Hardware**	Execution of Unreal Engine, AirSim and the application services.

Through these commitments, the company aims to combine technological innovation with sustainable development while fostering a responsible and inclusive working environment.

1.2 Project Presentation

1.2.1 Context and Problem Statement

The initial technical baseline was limited to a preliminary Unreal Engine scene developed by the company mentor, in which AirSim and a simulated drone had already been integrated. This prototype demonstrated that a drone could be controlled inside the virtual environment, but it did not yet implement a complete warehouse inspection use case. In particular, there was no unified workflow connecting rack selection, autonomous navigation, image acquisition, real-time supervision, visual analysis and structured result management.

The central problem of the internship was therefore to transform this initial simulation prototype into a coherent and testable logistics inspection demonstrator. This required defining a common representation of the warehouse, associating each inspection with a rack and a slot, implementing repeatable drone missions, retrieving several camera views and coordinating the different software components through a supervision application.

A second challenge concerned visual-data exploitation. Tag and pallet detection depend not only on the selected models, but also on drone stability, camera positioning, image quality, lighting conditions and annotation consistency. The project consequently had to address both the application architecture and the conditions under which reliable inspection images and training annotations could be generated.

1.2.2 Project Objectives

The objective of the project was to build an end-to-end chain linking the Unreal Engine/AirSim simulation, the drone mission logic, a back-end control layer, a front-end interface and visual-processing services. The solution had to make it possible to launch a rack-inspection mission, observe the drone's behaviour, retrieve images from the onboard cameras and present the information resulting from detection, OCR and barcode processing in the user interface.

More concretely, the project aimed to:

- control a simulated drone in a warehouse-like environment;
- organize repeatable movements according to racks and slots;
- position and stabilize the drone before image acquisition;
- display complementary camera views without multiplying external tools;
- detect logistics tags and extract text and barcode information;
- associate captures and results with their rack, slot, camera and timestamp;
- generate synthetic images and automatic annotations for computer-vision development;
- centralize communication between the simulation, image-processing services and the web application.

1.2.3 Functional Scope

The functional scope covers the generation and placement of logistics tags, the configuration of racks and slots, the implementation of rack-selectable

drone missions, the retrieval of images from cameras `cam0` and `cam1`, and the integration of UDP and Pixel Streaming mechanisms. It also includes the display of tag detections, OCR and barcode results, the organisation of captures by mission, and the generation of bounding boxes from simulation segmentation masks.

The current demonstrator remains simulation-oriented. It does not include complete autonomous navigation from a shared warehouse home position, simultaneous multi-drone operation or continuous synchronization with a physical warehouse. These elements are discussed as future developments rather than as completed functions.

1.2.4 Technical and Operational Constraints

The main constraints arose from the interaction between several technically different components: Unreal Engine for the 3D scene, AirSim for multirotor control and image acquisition, FastAPI for orchestration, React for user interaction, and network streams for real-time display. Strict consistency was required between the objects present in the scene, the rack configuration, the camera parameters and the information displayed in the interface.

The project also had to reuse the Unreal Engine/AirSim base provided at the beginning of the internship, remain testable when the full simulator was unavailable, prevent blocking AirSim calls from freezing the web server, and limit the accumulation of obsolete video frames. Finally, synthetic logistics data had to avoid exposing confidential real-world information while remaining visually credible.

1.2.5 Internship Assignments

My assignments focused on designing, extending and integrating the components required to turn the initial Unreal Engine/AirSim prototype into a structured logistics inspection demonstrator. The work included completing and organizing the warehouse environment, generating and positioning visual tags, implementing a mission script reusable across several rack configurations, and improving drone positioning for image acquisition.

The internship also involved developing the communication between the simulation, the FastAPI back end and the React supervision interface. Additional tasks concerned UDP and Pixel Streaming integration, camera and result management, visual tag analysis, and the preparation of segmentation-based bounding boxes and synthetic data for detection-model development.

These assignments were treated as interconnected parts of the same system. The objective was to preserve traceability between the selected rack,

the inspected slot, the drone position, the acquired images, the generated annotations and the results displayed to the operator.

Chapter 2

Methodology and Internship Progress

2.1 Project Methodology and Organization

2.1.1 Kickoff Meeting and Requirement Framing

The first phase consisted of examining the preliminary simulation provided at the start of the internship and clarifying the expected warehouse-inspection scenario. This analysis identified which components already existed, which ones had to be adapted and which ones still had to be implemented. It also established the main technical chain: a simulated warehouse, an AirSim-controlled drone, a back end responsible for orchestration and a web interface for supervision.

2.1.2 Task Planning

The work was divided into successive technical phases so that each block could be tested before being connected to the next one. The progression started with the Unreal Engine/AirSim environment and the warehouse configuration, continued with rack missions and camera acquisition, and then extended to back-end orchestration, supervision, streaming, visual processing and automatic annotation.

2.1.3 Working Method and Progress Tracking

Development followed short iterative cycles: understand one component, modify it, test it in isolation and then validate its behaviour within the complete chain. This approach was particularly important because a regression

Table 2.1: *Main development phases of the internship*

Phase	Main focus	Validation principle
1	Analysis of the Unreal Engine/AirSim prototype and scene structure.	Connection tests, object inspection and drone-control verification.
2	Rack, slot and tag configuration.	Consistency checks between JSON identifiers and Unreal Engine actors.
3	Drone mission and PID-based positioning.	Repeated movements to representative slots and position-error checks.
4	Camera acquisition and streaming.	Verification of image availability, freshness and browser display.
5	Back-end and supervision integration.	End-to-end session, mission and result workflows.
6	Visual analysis and automatic annotation.	OCR/barcode display, segmentation masks and generated dataset samples.

in the simulation, the network layer or the back end could block the entire mission workflow.

Progress was assessed through functional scenarios rather than only through unit-level execution. Examples included initializing a session, selecting a rack, moving the drone to a slot, acquiring both camera images, displaying a tag analysis and storing the corresponding mission outputs.

2.1.4 Collaboration, Configuration and Version Control

The project was organized around separate application services, Python mission scripts and centralized configuration files. Network parameters, actor names, rack definitions and important paths were kept outside the main control logic whenever possible. This separation improved readability and reduced the number of values duplicated across the codebase.

Version control and regular technical reviews were used to keep modifications traceable and to discuss implementation choices with the company mentor. Particular attention was paid to preserving compatibility with the existing Unreal Engine project while introducing new application services and

data-generation functions.

2.2 Internship Progress

This section presents the work carried out during the internship from the perspective of personal contributions. The guiding principle was to extend a preliminary Unreal Engine/AirSim prototype into an integrated experimental chain in which the drone mission, the application services, the visual streams and the generated results could be operated through a common workflow.

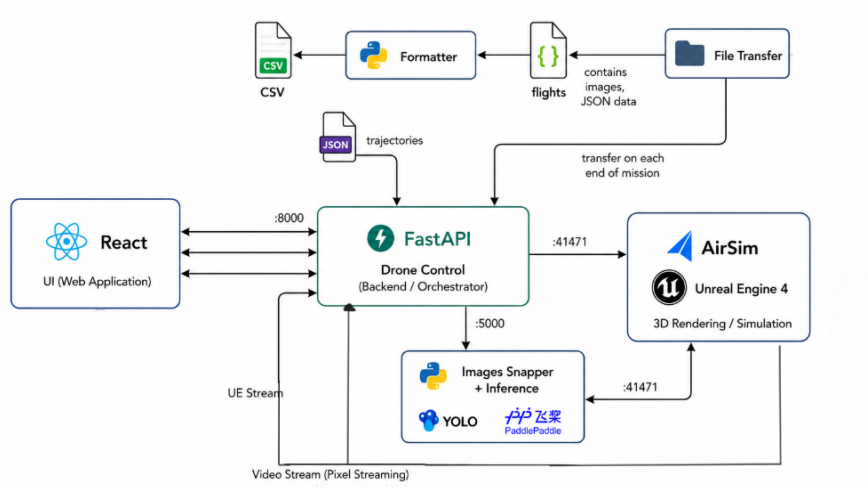


Figure 2.1: Overall architecture connecting the React interface, the FastAPI services, AirSim, Unreal Engine and the visual-processing components.

2.2.1 Initial Technical Baseline and Analysis

The elements available at the beginning of the internship were the partially developed Unreal Engine scene, the AirSim integration and a simulated drone capable of flying within the environment. The first technical task was therefore to understand the scene hierarchy, the drone configuration, the coordinate conventions and the constraints imposed by the existing project.

The analysis showed that the initial prototype could demonstrate flight in the scene, but that the logistics workflow still had to be structured. Rack definitions, slot associations, repeatable missions, application orchestration, supervision, multi-camera streaming, result traceability and automatic annotation were not yet connected as a single system.

The development path was then organised progressively: preserve the useful simulation base, consolidate the warehouse and mission logic, integrate

Table 2.2: *Initial assets and work carried out during the internship*

Component	Initial situation	Work during the internship
Unreal Engine environment	Preliminary 3D scene initiated by the company mentor.	Warehouse organisation, rack and tag integration, additional cameras, HUD and rendering adjustments.
AirSim and drone	AirSim already integrated and a simulated drone already present.	Rack-specific initialization, mission logic, live object-pose retrieval, PID positioning and synchronized captures.
Application layer	Technical blocks and experiments not yet organised as a unified workflow.	FastAPI orchestration, session and mission services, asynchronous execution and React integration.
Visual supervision	Initial or experimental views.	Multi-camera display, UDP/MJPEG transport, Pixel Streaming integration and mission-result consultation.
Computer vision data	Initial detection experiments and limited annotations.	Tag/OCR result display, segmentation-based automatic boxes, metadata capture and synthetic visual variants.

the application services and finally add visual analysis and dataset-generation functions.

2.2.2 Warehouse and Rack Configuration

The 3D scene, the back end and the supervision interface must share a common representation of the warehouse. A central JSON configuration therefore describes the available racks, their initialization information and the actors associated with each slot. This avoids duplicating actor names and rack definitions across several mission scripts.

The configuration has two different roles. At rack level, it can provide an initial pose file, an Unreal Engine marker actor or a configured position used to place the simulated drone before inspection. At slot level, it associates identifiers such as `A1` or `B3` with the current tag actor and the corresponding pallet actor. The final inspection target is then computed from the live tag pose returned by AirSim rather than from a hard-coded slot coordinate.

Table 2.3: *Role of the warehouse configuration*

Component	Use of the configuration
Interface	Display the available racks and selectable inspection elements.
Back end	Validate mission requests and schedule the selected rack configurations.
Mission script	Resolve the rack initialization method and the tag/pallet actors associated with each slot.
Simulation	Provide the live object poses used to compute inspection targets.
Results	Preserve the association between each capture, rack, slot, tag and pallet.

At rack level, the logical structure is a grid composed of three levels, labelled A, B and C, and four slots per level. An occupied slot references the visible tag and pallet actors used by the mission and the annotation pipeline.

Validation of the configuration was performed from global and local viewpoints. The global Unreal Engine view and the overhead view were used to check rack alignment, circulation areas and the relative position of the drone. Closer views were then used to verify pallet distribution, tag presence and the visibility of the target objects.

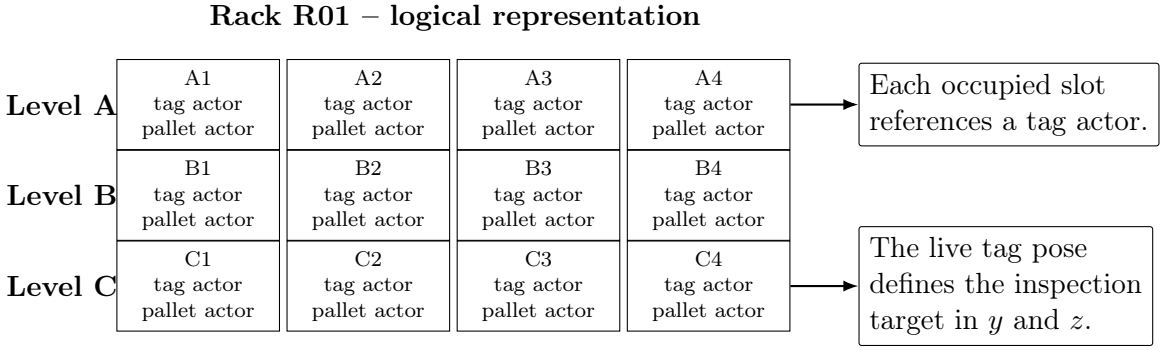
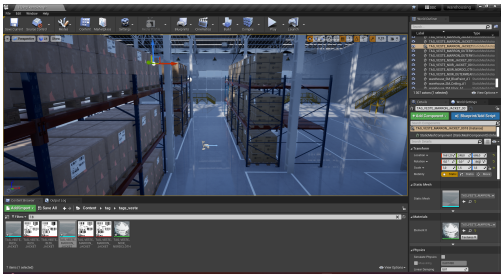
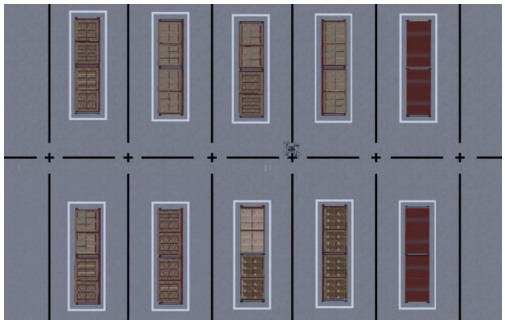


Figure 2.2: Logical representation of a rack with three levels and four slots per level.



Unreal Engine environment.



Overhead view of the hangar.

Figure 2.3: Global verification of the warehouse and spatial arrangement of the racks.



Rack inspection.



Pallet and tag inspection.

Figure 2.4: Local verification of racks, pallets and tags in the scene.

2.2.3 Drone Missions and AirSim Integration

The drone mission transforms the 3D scene into a structured inspection process. The mission script is generic across the configured racks, but one call to `run_mission(rack_id)` inspects one selected rack. When several racks are selected in the application, the orchestration layer must schedule the corresponding rack missions. In this report, the term *multi-rack support* therefore refers to the reuse of the same mission logic across several rack configurations, rather than to simultaneous inspection of several racks by one script invocation.

The implementation relies on the AirSim multirotor interface, which provides access to flight commands, estimated vehicle state, camera images and Unreal Engine object poses [2]. The principal mission parameters are summarised in Table 2.4.

Table 2.4: *Principal parameters of the rack-inspection mission*

Parameter	Configured value
Nominal speed	1.5 m/s
Position threshold	0.08 m
PID control interval	0.1 s
Main PID timeout	20 s
Assistance timeout	6 s
Post-positioning delay	8 s
Camera resolution	1920×1080 pixels

2.2.3.1 Mission Initialization and Rack Selection

Rack definitions are loaded from a central `racks_config.json` file. Each configuration provides the rack name, its initialization method, its orientation rule and the slot-to-actor associations. The initial drone pose can be obtained from a JSON pose file, an Unreal Engine marker actor or a configured position. For racks that require the opposite viewing direction, the yaw is set to 180°.

Before the inspection loop begins, the drone is placed at the rack-specific initial pose with `simSetVehiclePose`. This is a scenario-initialization operation in the simulator and is not counted as autonomous navigation performance. The API control is then enabled, the drone is armed and a take-off command is executed.

2.2.3.2 Slot Traversal Strategy

Each rack is represented by three levels and four slots per level. The mission processes the slots in the following order:

$$A1 \rightarrow A2 \rightarrow A3 \rightarrow A4 \rightarrow B4 \rightarrow B3 \rightarrow B2 \rightarrow B1 \rightarrow C1 \rightarrow C2 \rightarrow C3 \rightarrow C4.$$

This serpentine ordering limits unnecessary lateral travel when changing level: the following row starts from the side closest to the last inspected slot. Slots without an associated tag actor are skipped and should be distinguished from genuine navigation or capture failures in the mission statistics.

2.2.3.3 Target Pose Computation

For each occupied slot, the mission retrieves the live pose of the corresponding tag actor using `simGetObjectPose`. The flight target is therefore derived from the actual object position in the Unreal Engine scene rather than from a duplicated slot position in the Python mission. If the tag actor is absent or cannot be resolved, the slot is skipped and reported.

During the inspection of one rack, the drone keeps a constant coordinate along the axis perpendicular to the rack face. The lateral and vertical coordinates are aligned with the tag pose:

$$x_{\text{target}} = x_{\text{inspection plane}}, \tag{2.1}$$

$$y_{\text{target}} = y_{\text{tag}} + \Delta y, \tag{2.2}$$

$$z_{\text{target}} = z_{\text{tag}} + \Delta z. \tag{2.3}$$

In the current configuration, $\Delta y = 0$ and $\Delta z = -0.47\,\text{m}$. The vertical offset compensates for the displacement between the vehicle reference point and the inspection camera. Navigation values are interpreted in the North-East-Down coordinate convention used by AirSim, where the positive vertical axis points downward.

2.2.3.4 PID-Based Positioning

Movement toward each inspection position is controlled through a three-dimensional PID-based velocity command. At iteration k, the position error is:

$$\mathbf{e}_k = \mathbf{p}_{\text{target}} - \mathbf{p}_k.$$

The raw velocity command is computed as:

$$\mathbf{v}_k = K_p \mathbf{e}_k + K_i \mathbf{I}_k + K_d \frac{\mathbf{e}_k - \mathbf{e}_{k-1}}{\Delta t},$$

with $K_p = 0.70$, $K_i = 0.02$, $K_d = 0.35$ and $\Delta t = 0.1\,\mathrm{s}$. The integral vector is clipped between -1.5 and 1.5 on each axis to limit wind-up. The velocity norm is also capped according to the remaining distance, which progressively reduces the authorised speed near the target.

To avoid abrupt changes, the command sent to AirSim is smoothed with the previous velocity:

$$\mathbf{v}_k^{\mathrm{smooth}} = 0.8\,\mathbf{v}_{k-1}^{\mathrm{smooth}} + 0.2\,\mathbf{v}_k.$$

The controller is considered to have reached the target when the Euclidean position error is below 0.08 m. If the main loop exceeds 20 s, a slower AirSim position command is used for up to an additional 6 s. The controller and these practical stabilisation mechanisms were implemented to reduce overshoot and obtain camera views suitable for analysis, following standard feedback-control principles [4].

2.2.3.5 Stabilization, Image Acquisition and Mission Outputs

Once the positioning function has reached the configured threshold, the mission waits for an additional 8 s before image acquisition. The current implementation uses a fixed post-positioning delay; it does not continuously verify that the drone remains inside the 0.08 m threshold throughout the complete eight-second period. This distinction is important when interpreting the stabilisation behaviour.

The mission then acquires one RGB image from `cam0` and one from `cam1`. Each image is resized to 1920×1080 pixels and stored with camera metadata including the camera identifier, field of view, position and orientation. The files are organised by rack, mission timestamp, camera and augmentation category. Mission counters track successful captures and failed or skipped operations.

At the end of the rack inspection, the current implementation resets the simulated drone to its recorded rack-initial position before landing and disabling API control. As with the initial placement, this return uses a simulated pose reset and is not an autonomous return-to-home trajectory.

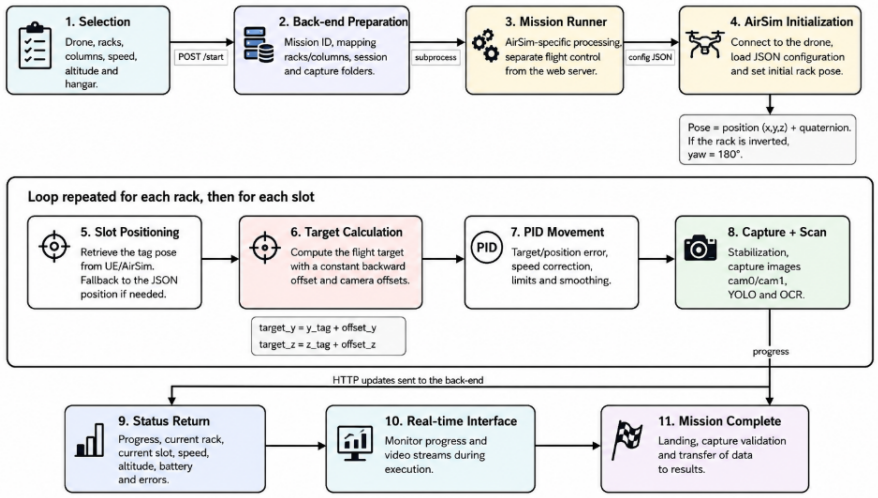


Figure 2.5: Sequential rack-inspection workflow from mission selection to capture, annotation and result storage.

2.2.4 Cameras, Streaming and Real-Time Rendering

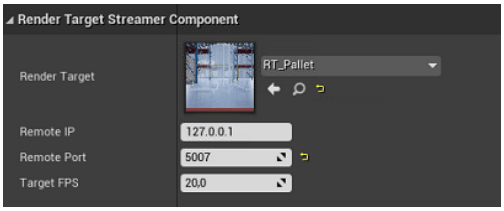
Visual streams support both human supervision and automatic image exploitation. Two AirSim cameras are dedicated to inspection, while Unreal Engine also provides an overhead view and a global browser-rendered view. Table 2.5 summarises the principal inspection-camera settings.

Table 2.5: *Configuration of the AirSim inspection cameras*

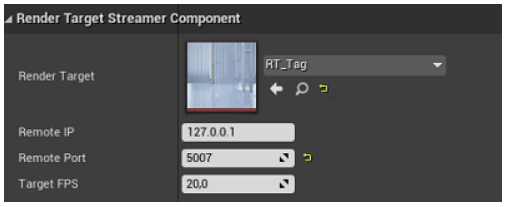
Camera	Resolution	Field of view	Principal use
`cam0`	1920 × 1080	20°	Close-up logistics-tag view, OCR and barcode reading.
`cam1`	1920 × 1080	90°	Wider pallet view and spatial context around the inspected slot.

The global Unreal Engine rendering is delivered to the browser through Pixel Streaming, whereas the specialised views are handled as low-latency streams. Additional Unreal Engine cameras were placed at coordinates corresponding to the AirSim inspection cameras. This preserves alignment with the control logic while providing direct access to Render Targets and a rendering path better suited to the Unreal Engine HUD [1].

The drone Blueprint flow relies on a few structural nodes. At startup, *Event BeginPlay* initializes the visualization widget, adds it to the screen



Render Target used for the pallet stream.



Render Target used for the tag stream.

Figure 2.6: Pallet and tag Render Targets used by the specialised views.

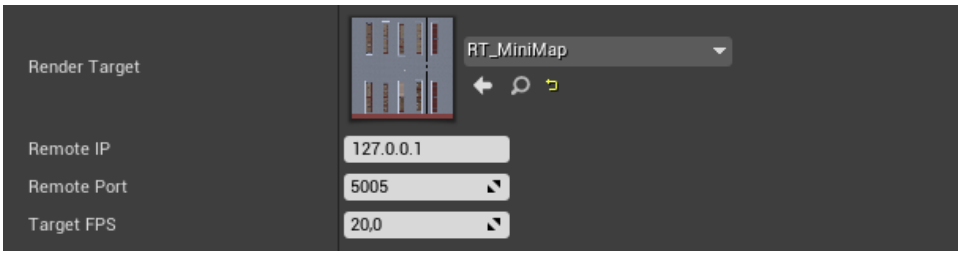


Figure 2.7: Render Target used for the overhead warehouse view.

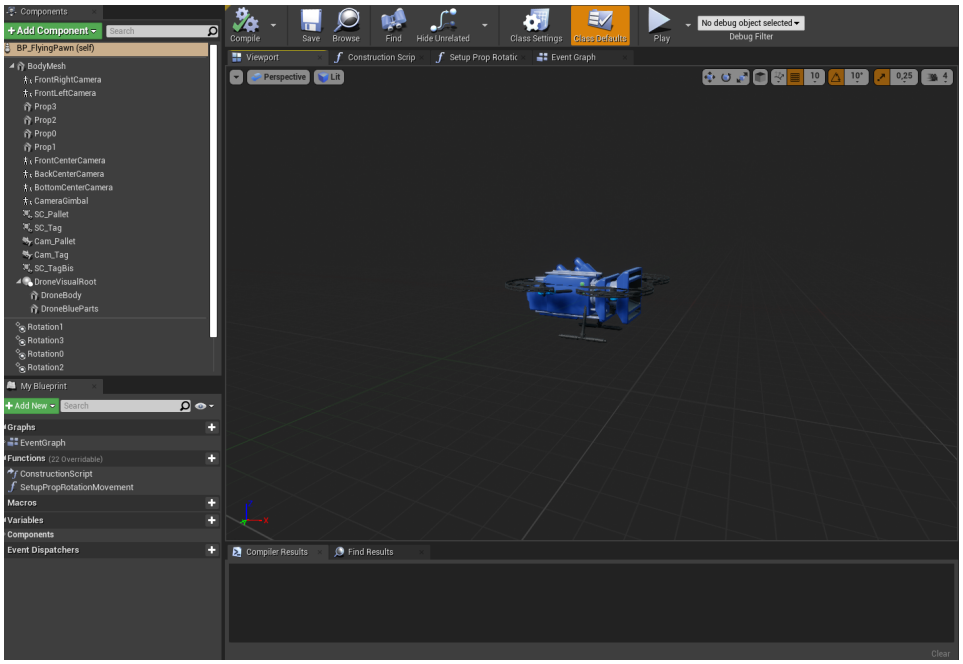
and links the player controller to the active camera. *Set Active* selects the required cameras, while *Set View Target with Blend* applies a smooth transition between the main, pallet and tag viewpoints. Propeller rotation does not affect perception, but improves the visual credibility of demonstrations.

2.2.4.1 UDP Streaming Performance Optimization

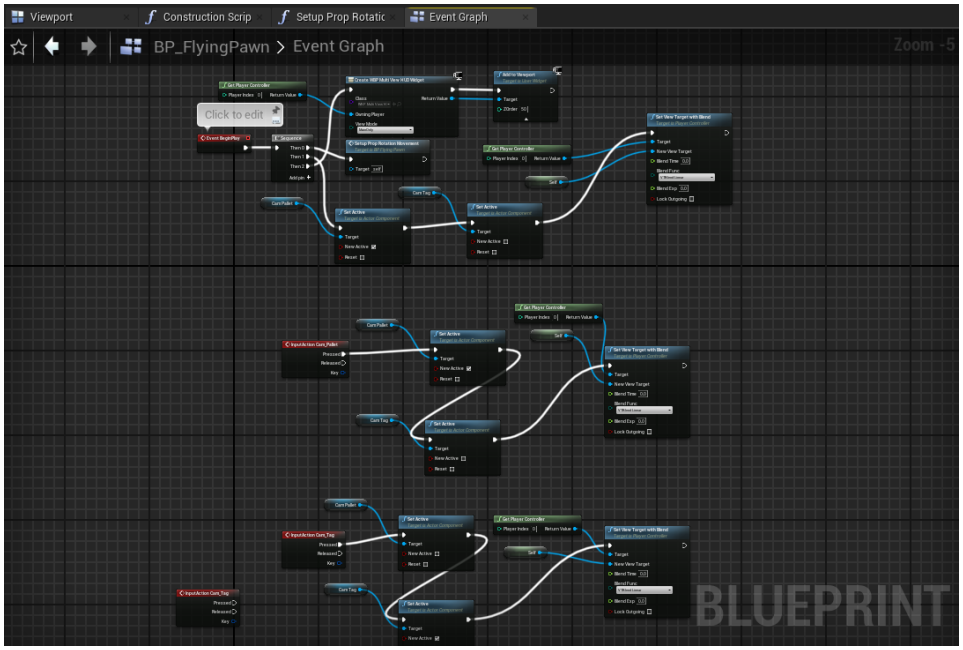
The specialised camera pipeline transmits encoded images through UDP. Since UDP does not guarantee delivery, ordering or retransmission [3], one encoded frame may be fragmented into several packets and must be reconstructed by the Python receiver. The implementation is designed to preserve low latency while preventing incomplete or obsolete data from accumulating.

Latest-frame priority The receiver keeps only the latest completed frame for display. A newly reconstructed frame replaces the previous one, and identifiers are used to reject obsolete frames that arrive significantly later than the current stream. This policy intentionally favours temporal freshness over exhaustive delivery.

Incomplete-frame management Partial frames are removed after a configurable Time-To-Live. Duplicate fragments and invalid packet metadata are ignored before reconstruction. These mechanisms prevent abandoned frames from consuming memory indefinitely and allow subsequent frames to be processed without waiting for retransmission.



Cameras reproducing the AirSim inspection viewpoints.



View switching.

Figure 2.8: Principal Blueprints associated with the drone.

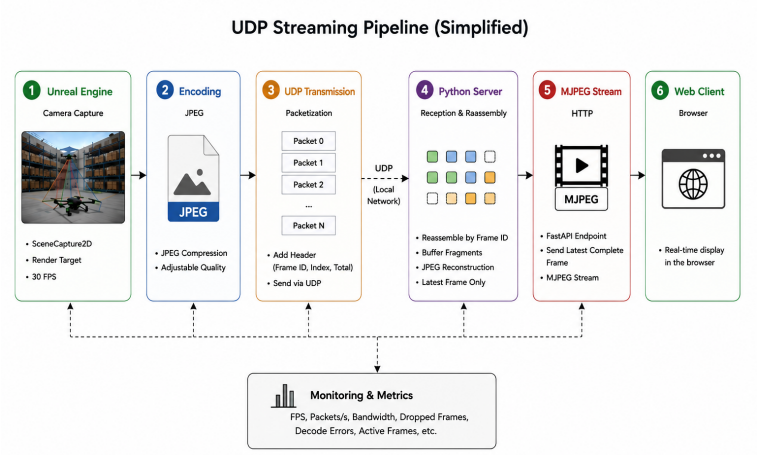


Figure 2.9: Simplified UDP-to-MJPEG streaming pipeline.

JPEG and MJPEG delivery When the reconstructed payload is already JPEG encoded, it can be forwarded to the HTTP/MJPEG response without an unnecessary decode–encode cycle. Images in another format are decoded with OpenCV and re-encoded with a configurable JPEG quality factor.

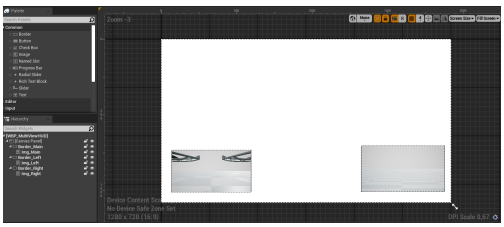
Frame-rate regulation and diagnostics The HTTP streaming loop applies a configurable target frame rate and sends an image only when a new completed frame is available. Monitoring endpoints expose packet counts, reconstructed frames, incoming bandwidth, frame rate, decoding errors, dropped frames, duplicate fragments and stream-reset events.

The main trade-off is that a frame becomes unusable when one or more required fragments are missing. The latest-frame policy reduces accumulated delay, but it deliberately discards images that are no longer temporally relevant. Quantitative comparisons of latency, CPU usage and packet-loss sensitivity remain necessary for a complete performance evaluation.

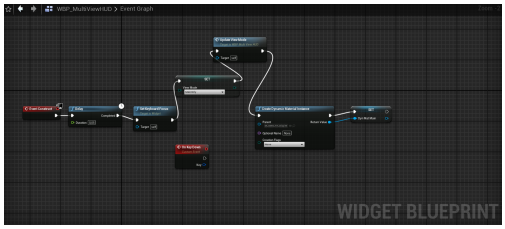
2.2.4.2 MultiView Interface in Unreal Engine

The multi-view widget combines the main view with two secondary areas dedicated to the pallet and tag cameras. It centralizes the display mode, keyboard focus and the association between Render Targets and the visible zones of the HUD.

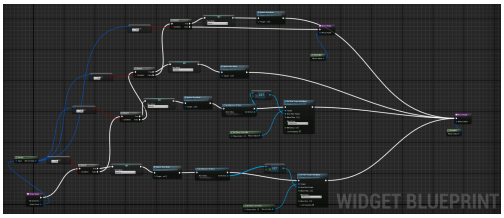
Event Construct initializes the display mode and gives keyboard focus to the widget. *On Key Down* compares the received key with the configured shortcuts and updates the *View Mode*. Finally, a *Switch on Enum* shows or hides the corresponding HUD zones and injects the Render Targets through



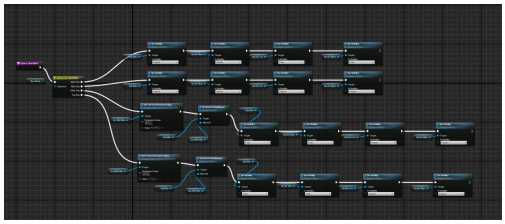
HUD visual organization.



Widget initialization.



Keyboard and view-mode selection.



Application of the selected mode.

Figure 2.10: Structure, initialization and interaction logic of the MultiViewHUD widget.

dynamic materials.

2.2.5 Structuring the Back-End Application

The back end became the coordination point between the interface, the simulation, the mission scripts and the visual-processing chain. A simple script-execution approach was no longer sufficient once missions had to be launched from the interface, tracked over time and connected to several services simultaneously.

The FastAPI application was therefore organised as an orchestrator. It receives user requests, validates the current session state, starts the required operation, communicates with AirSim, tracks mission progress and exposes captures and results to the interface. This separation keeps display concerns in React and technical execution in the back end.

The application is divided into specialised routers. The `ops` router handles the session lifecycle, flight commands, captures and drone-state changes. The `mission` router manages autonomous launches, progress tracking and result consolidation. The `flights` and `csv` routers provide access to archived missions and logistics outputs. This organisation limits side effects and makes each functional domain easier to maintain.

A staged initialization sequence prevents a mission from being launched before the required components are ready. The sequence includes connection verification, camera-parameter preparation, flight-controller initialization and a final *Ready* state. The objective is not to claim safety certification, but

to ensure that the software prerequisites have completed successfully before mission execution is enabled.

The communication layer also supports a Mock Mode. When AirSim is unavailable, a virtual drone state can be used to validate the interface and mission workflow without launching the 3D simulation. Blocking AirSim operations are executed in worker threads so that the FastAPI event loop remains available for status and interface requests.

Table 2.6: *Main exchanges coordinated by the back end*

Source	Destination	Exchange	Purpose
React	**FastAPI**	**HTTP/JSON**	Session actions, rack selection, mission launch and status queries.
FastAPI	**Mission script**	**Function or worker execution**	Start a configured inspection and collect its status.
Mission script	**AirSim**	**AirSim RPC**	Pose queries, flight commands, camera images and segmentation masks.
Unreal Engine	**Python stream server**	**UDP/JPEG fragments**	Low-latency specialised camera streams.
FastAPI	**React**	**HTTP/MJPE and JSON**	Live views, progress, captures and structured results.

This architecture preserves a clear control flow: the interface expresses a user action, the back end verifies and orchestrates it, and the specialised services execute the simulation, streaming or visual-processing operations.

2.2.6 Evolution of the Supervision Interface

A significant part of the internship focused on the web interface, since it is the operator's natural entry point into the system. The first version mainly played a demonstration role: it made it possible to test certain views, display information and verify that the first exchanges with the simulation worked. This basis was useful to validate the general principle, but it remained limited

for operational use, as it did not yet clearly structure control actions, mission tracking and result reporting.

The consolidated version is structured as a supervision demonstrator. The user can start a session, select a rack, follow the drone status, display the camera streams, consult captures and access results from a single environment. This choice follows an integration logic: the interface should not only display information, but also provide continuity between mission preparation, execution and exploitation of the produced data.

The evolution took place in several stages: a first version validated display and basic exchanges with the simulation, a second grouped the essential control and consultation functions, and the consolidated version was designed as a supervision workspace.

Table 2.7: *Functional evolution of the interface*

Version	Main Role	Added Value
Initial prototype	Test the first views and validate the first exchanges with the simulation.	Useful for experimentation, but still limited for full use and mission tracking.
Intermediate version	Group the main actions into a single working screen.	Added control, mission, inventory and results pages, reducing the need for separate actions.
Consolidated version	Provide a clearer supervision environment for integrated demonstrations and tests.	Better screen organization, video stream integration, mission tracking, captures and archived results.

This evolution also made it possible to better separate responsibilities. The interface was not meant to contain the entire project logic; it was meant to let the user trigger actions, monitor system status and understand the results. Heavier processing was therefore left to the back end and specialized scripts.

In practical terms, the interface works as a supervision layer. It sends requests to the back end, displays the responses received and updates the views according to the current system state. When a mission is launched, it transmits the user's choices and then regularly queries the mission status to

display progress, the current rack, captures and results.

The consolidated version provides a more direct and coherent user journey. The operator no longer needs to know the internal organization of the project: they select an action, observe the visual feedback and consult the results. This also makes the interface easier to evolve, since a new feature can be added as a new screen or button linked to a back-end action.

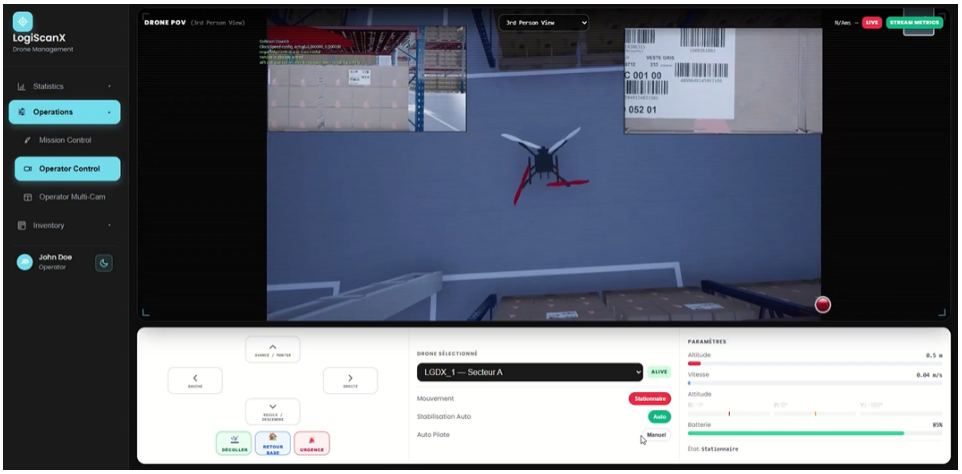


Figure 2.11: Previous version of the supervision interface.

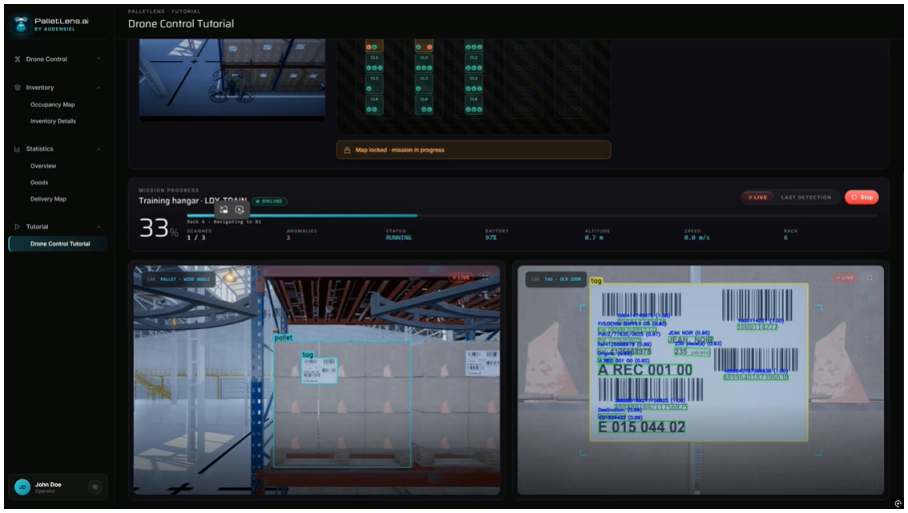


Figure 2.12: Final version of the supervision interface.

2.2.7 Visual Tag Detection, OCR and Barcode Recognition

The visual inspection chain does not stop at image transmission. Once the drone has reached a scanning position, the tag-oriented image is processed to locate the complete logistics tag and extract the information printed on it. The resulting data can then be displayed to the operator and associated with the inspected rack and slot.

The first operation is **tag detection**: a bounding box identifies the complete label in the wider camera image. The detected region is then cropped or enlarged. Inside this region, **text detection** locates the areas likely to contain characters, while **text recognition** converts these image regions into strings. Barcode regions follow a separate decoding process. Keeping these operations distinct is important because a correctly located region can still produce an incorrect OCR result.

Recognized strings, decoded barcode values and confidence scores are superimposed on the detailed view. This visual overlay allows the operator to compare the automatically extracted information with the original label content. The secondary camera views remain visible to preserve the spatial context of the inspection.

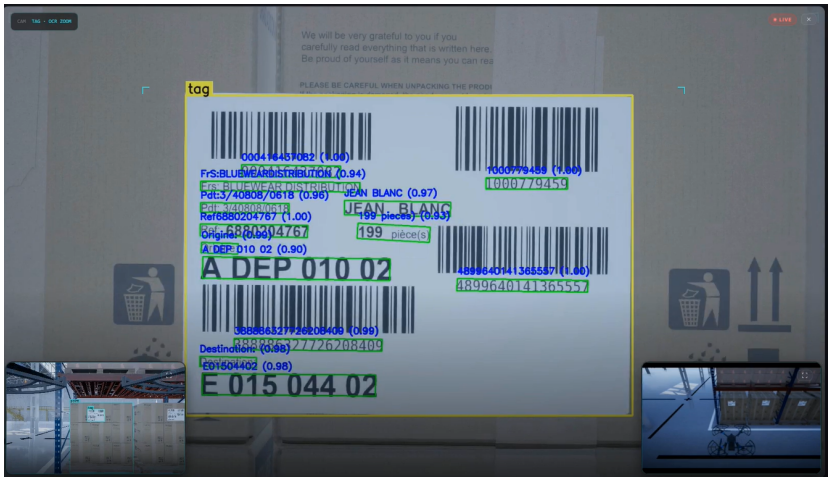


Figure 2.13: Visual analysis of a logistics tag in the supervision interface. The outer box identifies the complete tag, while internal regions show detected text and barcodes together with recognised values and confidence scores.

Each processed capture remains linked to its mission context: rack identifier, slot identifier, acquisition timestamp and camera identifier. This association ensures traceability between the visual observation, the location represented in the simulation and the result displayed in the interface.

2.2.8 Synthetic Dataset and Automatic Annotation Pipeline

Part of the work carried out during the internship contributes to a scientific-paper project concerning synthetic logistics data and automatic annotations. The report describes the implemented pipeline and its role in the platform while avoiding the disclosure of any unpublished methodological contribution that may require separate publication.

2.2.8.1 Synthetic Logistics Tags and Confidentiality

Real logistics labels may contain internal references, commercial information, product codes or supply-chain data. Reusing such labels directly in a simulation would raise confidentiality and compliance concerns. The project therefore relies on fictitious but visually credible metadata to generate logistics tags that reproduce the structure of real labels without exposing operational information.

The generated two-dimensional tags are converted into textures or materials and associated with pallet or rack actors in Unreal Engine. They can consequently be observed from the drone cameras under the same perspective, distance and lighting constraints as the other objects in the scene.

2.2.8.2 RGB Capture and Metadata Acquisition

For each inspected slot, the mission captures RGB images from both AirSim cameras. In addition to the image itself, the acquisition function stores camera metadata: identifier, resolution, field of view, position and orientation. The poses of the current tag and pallet actors are also collected when available. These data preserve the relationship between the two-dimensional image and the corresponding three-dimensional simulation state.

The output hierarchy separates rack identifiers, mission timestamps, cameras and augmentation categories. This structure improves traceability and makes it possible to reconstruct the conditions under which each sample was generated.

2.2.8.3 Segmentation-Based Automatic Annotations

The training-data pipeline is distinct from the inference pipeline used to display OCR and barcode results. During inference, a detector estimates the location of an object from an RGB image. During dataset generation, the simulation provides target-specific segmentation masks that serve as ground truth.

Before a segmentation capture, the visible scene objects are reset to segmentation identifier 0. The target pallet or tag actor is then assigned a dedicated identifier. AirSim captures the resulting segmentation image from `cam0` and `cam1`, producing a target-only mask. Pixels belonging to the target are isolated and used to calculate the corresponding bounding box automatically.

This approach reflects the actual visible projection of the object in the camera image. It accounts for perspective, object pose and partial visibility more directly than a box estimated only from theoretical dimensions or three-dimensional coordinates. After capture, the segmentation identifiers are restored to their neutral state.

The RGB images, target masks, camera metadata, object poses, rack identifier and slot identifier are then transmitted to the dataset service, which creates the annotated training sample.

$$\begin{gathered}\text{RGB images} + \text{target segmentation masks} \\ + \text{camera metadata} + \text{rack/slot metadata} \\ \longrightarrow \text{automatically annotated dataset sample.}\end{gathered}$$

2.2.8.4 Lighting and Orientation Variants

To increase visual diversity, the mission generates several variants for each occupied slot. The tag can be tilted by a random angle, the directional-light orientation can be changed, and three illumination scenarios can be produced: base, low and high. When runtime lighting changes do not create a sufficiently visible difference in the captured image, optional photometric transformations adjust contrast, brightness and gamma.

The complete sequence produces up to twelve RGB variants per camera for one occupied slot: one base image, one tag-rotation image, one light-rotation image and three images for each of the three lighting groups. With two cameras, this represents up to twenty-four RGB images per occupied slot, in addition to the segmentation masks and metadata used for annotation.

2.2.8.5 Quality Controls and Current Limits

The pipeline checks whether camera responses are valid, whether actor names can be matched by AirSim and whether the lighting variants produce a measurable luminance difference. Failures are reported instead of silently creating incomplete samples. Nevertheless, the generated data remain dependent on the quality of the Unreal Engine assets, the actor-naming conventions and the visual gap between simulation and real warehouse images. A separate

Table 2.8: *Synthetic capture categories generated for each occupied slot*

Category	Tag rotation	Light rotation	Illumination
Base	**No**	**No**	Base
Tag rotation	**Yes**	**No**	Base
Light rotation	**No**	**Yes**	Base
Lighting variants	**No**	**No**	Base, low and high
Lighting + light rotation	**No**	**Yes**	Base, low and high
Lighting + light + tag rotation	**Yes**	**Yes**	Base, low and high

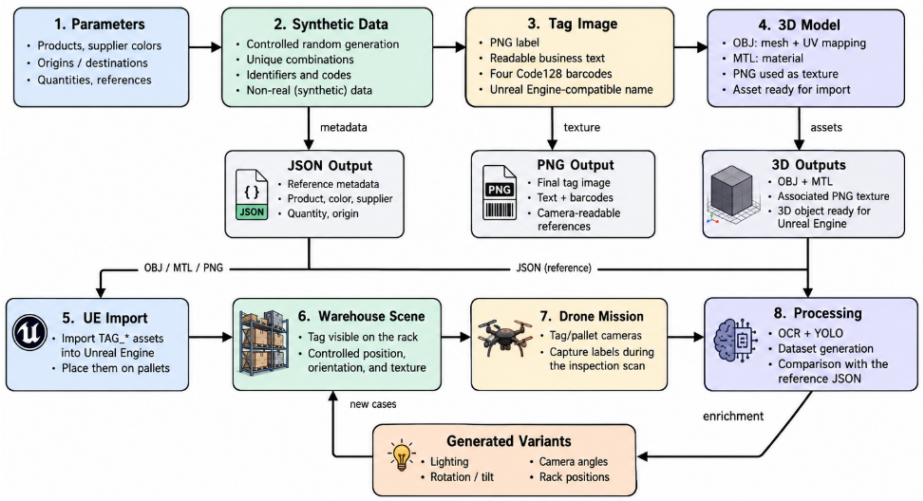


Figure 2.14: Simplified generation and exploitation chain for synthetic logistics tags and automatic annotations.

quantitative study is therefore required to measure the effect of these synthetic variants on model accuracy and generalisation.

2.2.9 Functional Validation of the Integrated Workflow

The completed platform was validated through end-to-end functional scenarios. The purpose of these tests was to verify that the components exchanged the expected information and that a complete inspection sequence could be executed in simulation. Table 2.9 summarises the principal checks.

Table 2.9: *Functional validation of the integrated workflow*

Function	Validation action	Observed output
Rack configuration	Load a selected rack and resolve its slots.	Rack, slot, tag and pallet identifiers available to the mission.
Mission positioning	Retrieve a live tag pose and command the PID movement.	Drone aligned with the selected slot within the configured threshold.
Dual-camera acquisition	Request `cam0` and `cam1` after positioning.	Two RGB images and their camera metadata stored in the mission hierarchy.
Visual analysis	Process the detailed tag view.	Tag box, OCR regions, barcode values and confidence scores displayed.
Automatic annotation	Capture target-only segmentation masks.	Bounding boxes and sample metadata generated without manual framing.
Supervision	Launch and monitor a mission through the application.	Progress, camera streams, captures and results accessible from the interface.

This validation is functional rather than statistical. The report does not claim measured OCR accuracy, guaranteed UDP latency or a formal PID-performance bound. Such claims would require repeated quantitative experiments with clearly defined datasets, hardware and network conditions.

2.2.10 Summary of Personal Contributions

The following table summarises the principal contributions completed during the internship.

Table 2.10: *Summary of personal contributions*

Area	Work performed	Current limitation or next step
Warehouse scene	Rack and slot organisation, tag/pallet actor associations and visual consistency checks.	Simplify configuration editing and add automated actor-validation tools.
Drone mission	Generic rack mission, serpentine slot traversal, live tag-pose retrieval, PID velocity control and synchronized captures.	Replace simulated pose resets with continuous navigation and distinguish empty slots from failures.
Application back end	FastAPI orchestration, staged initialization, asynchronous AirSim execution and mission-state exposure.	Strengthen interruption recovery and long-duration robustness.
Supervision interface	Rack selection, mission control, camera views, capture consultation and result display.	Conduct user-oriented ergonomic evaluation and improve error feedback.
Streaming	UDP frame reconstruction, latest-frame priority, TTL management, MJPEG delivery and diagnostics.	Measure latency, bandwidth, CPU usage and packet-loss sensitivity.
Visual processing	Integration of tag boxes, OCR text regions, barcode values and confidence overlays.	Quantitatively evaluate recognition accuracy on controlled and varied conditions.
Synthetic dataset	Camera metadata, target-only segmentation masks, automatic boxes, tag rotation and lighting variants.	Evaluate the simulation-to-real gap and the effect on model training.

Chapter 3

Feedback and Reflection

3.1 Reflection

3.1.1 Project Contributions

The principal value of the project lies in the integration of components that are often developed separately: a 3D simulation, multirotor control, camera acquisition, network streaming, computer vision, dataset generation and a web supervision application. The resulting demonstrator provides a traceable chain from rack selection to drone positioning, image acquisition, visual analysis and result storage.

The project also demonstrated the importance of modularity. Separating the interface, back-end routers, mission logic, stream receiver and dataset service made it possible to test one layer without reproducing all of the internal logic of the others. The Mock Mode is a direct example of this principle because it enables interface tests when the simulator is unavailable.

3.1.2 Analysis of Technical Choices

Unreal Engine and AirSim were retained because the internship had to build on the existing simulation base and because AirSim exposes a practical interface for multirotor state, flight commands, RGB images, segmentation images and object poses. FastAPI provided an appropriate orchestration layer for HTTP endpoints and asynchronous application logic, while React allowed the operator workflow to be grouped into a single supervision interface.

UDP was used for specialised camera views because temporal freshness was more important than guaranteed delivery of every frame. This choice introduces packet-loss and reconstruction constraints, which were addressed through frame identifiers, TTL cleanup and latest-frame priority. Pixel

Streaming was kept for the global Unreal Engine view because it exposes the rendered scene directly in the browser.

The PID controller was selected to provide progressive velocity corrections and explicit control over the approach to each tag. Its gains and practical limits are implementation parameters rather than universally optimal values. Their suitability should therefore be interpreted within the tested AirSim scene and mission configuration.

3.1.3 Difficulties Encountered and Solutions

Coordinate and pose consistency The rack configuration, Unreal Engine actors and AirSim state had to refer to the same physical locations. The final approach reduced duplicated coordinates by resolving the current tag pose directly from the scene and using configuration files mainly for rack and actor associations.

Stable camera positioning Direct position commands could produce residual movement or overshoot near a tag. A PID velocity controller, speed limitation, integral clipping, command smoothing and a slower assistance phase were combined to improve convergence before capture.

Video-frame accumulation A conventional queue could progressively increase the displayed delay when the consumer was slower than the producer. The stream receiver was therefore designed to keep the latest completed frame and discard obsolete or incomplete data after a timeout.

Blocking simulator calls Long AirSim operations could prevent the web server from responding to status requests. These operations were moved to worker threads, while the API retained the role of exposing state and results.

Automatic annotation Manual bounding-box generation was costly and potentially inconsistent. Target-specific segmentation masks were introduced so that boxes could be derived from the visible object projection produced by the simulator.

3.2 Skills Acquired

3.2.1 Technical Skills

The project strengthened my skills in Python, FastAPI, React integration, network streaming, image processing, Unreal Engine Blueprints and AirSim control. It also required the implementation of a feedback controller, the manipulation of vectors and quaternions, the management of NED coordinates, and the organisation of image datasets with camera and object metadata.

3.2.2 Methodological Skills

The internship required analysing an existing technical base before modifying it, isolating the source of failures across several services, validating features progressively and documenting limitations without overstating the results. These skills are directly connected to the software-engineering, computer-vision and robotics components of the master's programme.

3.3 Current Limitations

The current implementation has several clearly identified limitations:

- one call to the mission script inspects one rack; multi-rack execution requires orchestration of several calls;
- the initial placement and final return use simulated pose resets rather than continuous autonomous navigation;
- the eight-second stabilization phase is a fixed delay and does not continuously verify the position threshold;
- empty slots and technical failures should be represented by separate mission counters;
- UDP streams can lose a complete frame when one required fragment is missing;
- runtime lighting modifications may depend on the Unreal Engine/AirSim build, which is why a photometric fallback is available;
- the report provides functional validation but does not yet contain a full quantitative benchmark of PID response, streaming latency, OCR accuracy or detection performance;
- the visual gap between synthetic and real warehouse images has not yet been measured on a sufficiently large real dataset.

3.4 Future Project Evolution

A first evolution would replace rack-specific simulated pose resets with continuous navigation from a common home position. The mission scheduler could then optimise the order of several selected racks and, in a later stage, allocate work to multiple simulated drones while considering collision avoidance and shared state.

A second direction concerns quantitative evaluation. Repeated trajectories could be used to measure convergence time, residual position error and overshoot. The streaming system could be assessed through latency percentiles, frame-completion rate, bandwidth and resource consumption. OCR and detection results could be evaluated against a manually verified test set.

Finally, the synthetic-data pipeline could be integrated into a closed improvement loop. Errors observed on the detection and OCR models could guide the generation of new lighting, pose, distance and occlusion conditions. The resulting samples could then be used to retrain the models and measure whether the added variability improves robustness.

General Conclusion and Outlook

This internship led to the development of an integrated simulation and visual analysis demonstrator for drone-based logistics inspection. Starting from a preliminary Unreal Engine environment in which AirSim and a simulated drone were already present, the work structured the warehouse scenario and connected rack configuration, mission execution, camera acquisition, application orchestration, real-time supervision and visual-data generation.

The mission implementation retrieves live tag poses from the scene, follows a serpentine slot order and uses a smoothed three-dimensional PID velocity controller to align the drone before capture. The two inspection cameras provide complementary views for tag reading and pallet context. The supervision chain combines UDP/MJPEG streams for specialised views with Pixel Streaming for the global Unreal Engine rendering. Visual results include tag detection, OCR and barcode overlays, while target-specific segmentation masks provide automatic annotations for synthetic dataset generation.

The project should be considered a functional research and demonstration platform rather than a finished industrial inspection system. Its main remaining needs are quantitative performance evaluation, continuous autonomous navigation between racks, stronger failure recovery and validation against real warehouse images. Nevertheless, the completed work provides a coherent foundation for studying the relationship between drone positioning, image acquisition, visual recognition and synthetic-data generation.

Bibliography

[1] Epic Games, *Unreal Engine 4.27 Documentation*, 2021.

[2] S. Shah, D. Dey, C. Lovett and A. Kapoor, "AirSim: High-Fidelity Visual and Physical Simulation for Autonomous Vehicles," *Field and Service Robotics*, 2017.

[3] J. Postel, *User Datagram Protocol*, RFC 768, Internet Engineering Task Force, 1980.

[4] K. J. Åström and R. M. Murray, *Feedback Systems: An Introduction for Scientists and Engineers*, Princeton University Press, 2008.

[5] G. Bradski, "The OpenCV Library," *Dr. Dobb's Journal of Software Tools*, 2000.

[6] S. Ramírez, *FastAPI Documentation*.

[7] React Team, *React Documentation*.